

Technical Challenge: ORB Feature Matching, Result, and Analysis

Quy Le

October 2025

1 Objective

The goal of this technical challenge is to implement a feature matching pipeline using ORB algorithm. To that end, I implement a single Python script with OpenCV library to visualize matched keypoints between 2 images. Particularly, this involves detecting keypoints, computing descriptors, matching features, and analyzing the result.

The code was tested on 3 images pairs, including synthetic images from the AirSim simulator and real-world forest images, to evaluate its performance under different conditions.

2 Feature Matching Pipeline

Here are the steps in my pipeline:

1. **Detect & Compute:** Keypoints and its descriptors are found using `cv.ORB.detectAndCompute`, set to find up to 2000 features per image.
2. **kNN Matching:** A brute-force matcher (`cv.BFMatcher`) with hamming distance was used to find the $k=2$ best matches for each descriptor.
3. **Ratio Test filtering:** Lowe's ratio test was applied to the 2 matches. A match was kept only if the distance from the best match is significantly better than the second-best. This not only filters out ambiguous matches, but also reduces the amount of matches to be processed in the next filtering step.
4. **Outlier rejection:** The "good" matches from the ratio test are further filtered using RANSAC. RANSAC is used to estimate the Essential Matrix (`cv.findEssentialMat`), from the matched keypoints, while rejecting outliers that does not fit into the expected geometry model. This step is important, since it discards the outliers and only keep the inliers, a.k.a geometrically-correct matches.

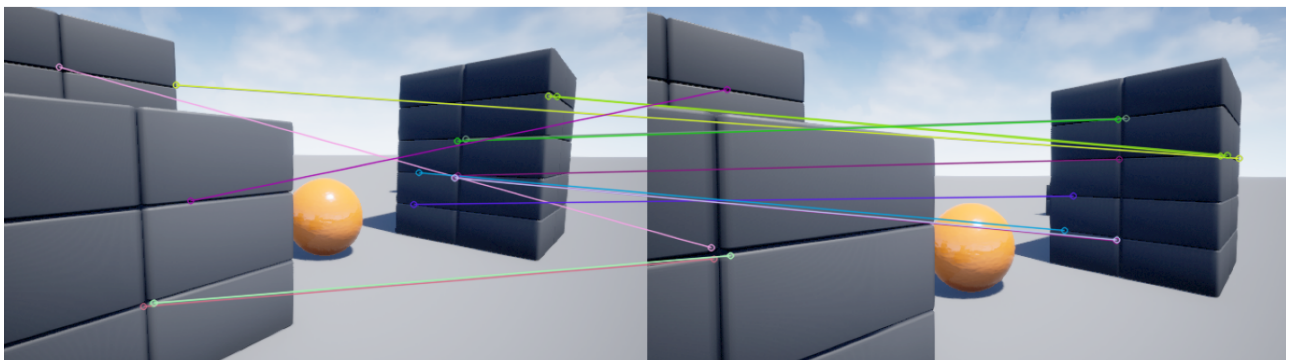


Figure 1: Outliers matches in image pair of block environment in AirSim

3 Results and Analysis

The pipeline was run on 3 image pairs.

3.1 Pair 1: AirSim - Simple blocks environment

This pair simulates a drone moving slowly forward in the high-contrast “Blocks” environment.

- **Quantitative Metrics:**

- Keypoints (Image 1): 599
- Keypoints (Image 2): 461
- Matches (After Ratio Test): 116
- Inliers (After RANSAC): 100 (86.2% inlier ratio)

- **Visualization:**

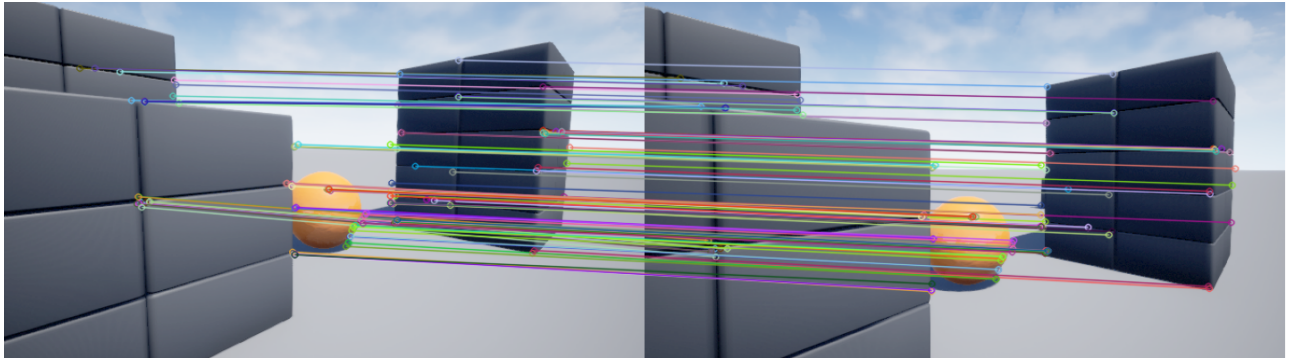


Figure 2: Matches between two sequential images in the simulated blocks environment

- **Analysis:** The low keypoints count (599/461) are the most notable, given the 2000-feature cap. This is because while the blocks environment has sharp corners, large portion of it is just textureless, which ORB correctly ignores. However, the detected features are clearly high-quality and unique. This is proven by the **86.2% inlier ratio**.

3.2 Pair 2: AirSim - Mountain landscape environment

This pair simulates a drone moving slowly forward in the mountainous forest environment.

- **Quantitative Metrics:**

- Keypoints (Image 1): 1972
- Keypoints (Image 2): 1978
- Matches (After Ratio Test): 204
- Inliers (After RANSAC): 194 (95.1% inlier ratio)

- **Visualization:**

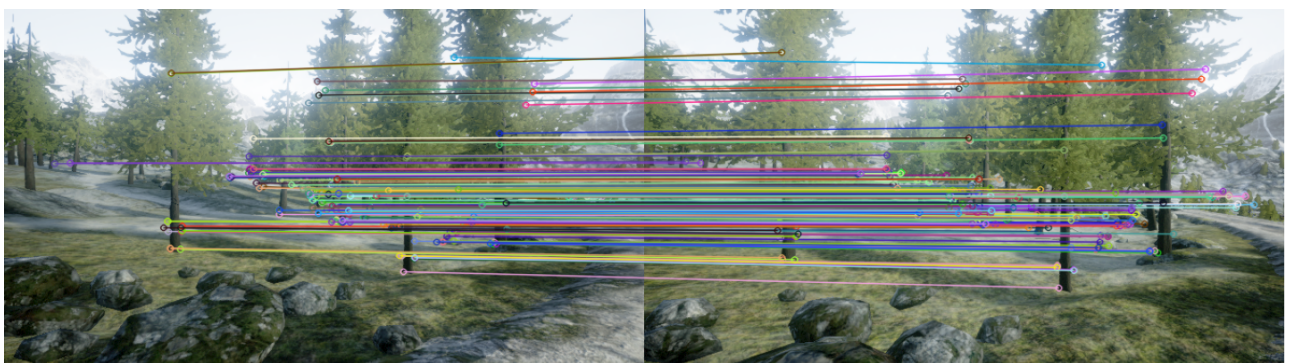


Figure 3: Matches between two sequential images in the simulated mountain environment

- **Analysis:** This environment has many textures, allowing ORB to detect max number of keypoints (1970/2000). There is an interesting drop in the number of matches from 1970 keypoints to 204 matches after the ratio test. This indicates that natural landscapes have many repetitive textures (e.g. foliage), creating many ambiguous matches that the ratio test correctly filter out. The **95.1% inlier ratio** indicates that the filtering was so effective, leaving only small number of outliers.

3.3 Pair 3: Real-World Forest (Stereo Pair)

This pair simulates a drone moving slowly forward in the mountainous forest environment.

- **Quantitative Metrics:**

- Keypoints (Image 1): 1963
- Keypoints (Image 2): 1965
- Matches (After Ratio Test): 495
- Inliers (After RANSAC): 482 (97.4% inlier ratio)

- **Visualization:**

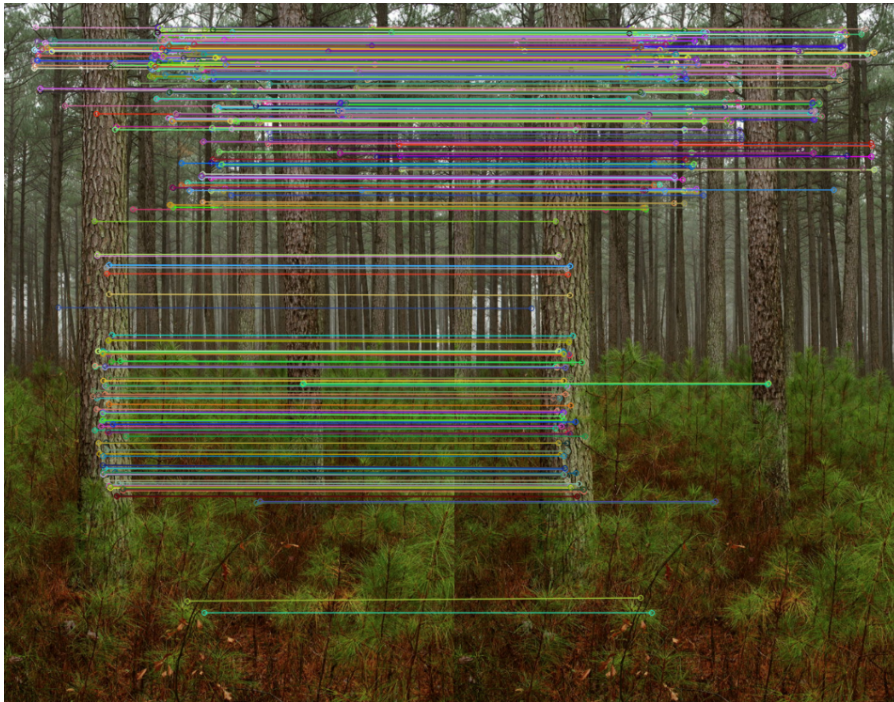


Figure 4: Matches between two sequential images in the simulated mountain environment

- **Analysis:** The real-world forest is also rich in features, maxing out the keypoints detection (1960/2000). The notable point is **97.4% inlier ratio**. This result is the result of a stereo image pair, where the viewpoint change is small compared to sequential images from a moving drone.

4 Extension: Visual Odometry

To challenge myself and demonstrate how feature matching can be used for autonomous rone, I extend the challenge to a simple Visual Odometry. To that end, I implement a Python ROS node that listen to the real-time camera topic publishers and estimate the path in real-time.

- **Method:**

1. The feature-matching pipeline was run on consecutive frames from a drone flying a pre-defined path in AirSim.

2. The Essential Matrix E was calculated from the inlier matches.
 3. `cv2.recoverPose()` was used to extract the camera's Rotation (R) and Translation (t) between each frame.
 4. These (R, t) transformations were chained together to estimate the drone's full 3D trajectory.
- **Result:** The plot below compares the trajectory estimated by my VO pipeline (red) against the simulation's ground truth (green).

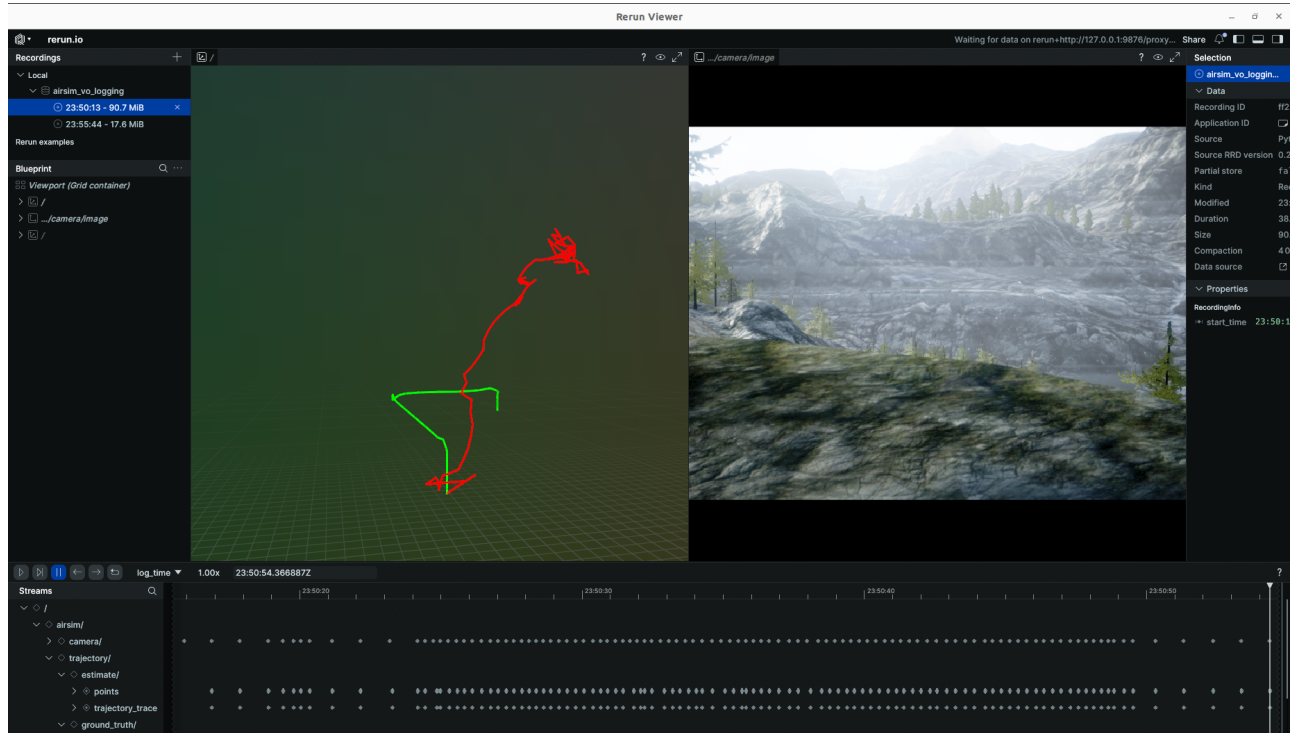


Figure 5: Estimate path (red) vs ground truth path (green)

- **Analysis:** The estimate path does not follow the ground truth at all. This comes from the drift due to scale ambiguity and error accumulation, which is expected from monocular VO. Or maybe something is wrong with my VO implementation. This is my first time doing this in 3D, since working with cars assume that the altitude is constant. But this is a good starting point!

5 Conclusion

This implementation indicates a robust pipeline for ORB feature matching. The key takeaway here is that the raw match counts are misleading. Filtering (ratio test + RANSAC) is essential for getting high-quality and geometrically-correct inlier matches. The Visual Odometry extension is fun and could be a good starting point to implement a better state estimation pipeline.